C/C++-Kurs für Kinder ab der 5. Klasse

Jürgen ‘‘George’’ Sawinski

25. Juli 2026

Zusammenfassung

Dieser Kurs ist gedacht für Kinder ab der Grundschule mit guten mathematischen Fähigkeiten und dem Drang, programmieren zu lernen. Er entstand, nachdem die Söhne des Autors ihn über ein Jahr beknieten, ein Spiel zu entwickeln.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation zu in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

1 Etwas Mathematik **3**

2 Hallo Welt! **5**

3 Zahlen **7**

4 Fibonacci **8**

5 Reihen **9**

A Software **10**

B Leitfaden **11**

B.1 Tastenkombinationen 11

B.1.1 Sonderzeichen 11

B.1.2 Referenz 12

B.2 C 12

B.2.1 Grundlagen 12

Vorwort

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch **DeepSeek**, ein KI-gestütztes Werkzeug, unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

– Jürgen ‘‘George’’ Sawinski, 25. Juli 2026

1 Etwas Mathematik

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt ‘‘**Bit**’’ aneinandergereit. Am gebräuchlichsten dabei ist das ‘‘**Byte**’’ (siehe Abb. 1), das aus 8 Bit besteht.

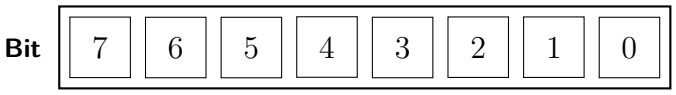


Abbildung 1: Ein Byte mit den Bit-Positionen 0 bis 7.

Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

$$2^b$$

wobei b die Nummer des Bits ist. Das ist eine sogenannte ‘‘Potenz’’, eine mathematische Operation die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die ‘‘2’’ b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn ihr genau hinschaut, dann merkt ihr, dass, um in die nächste Zeile zu kommen, ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 2).

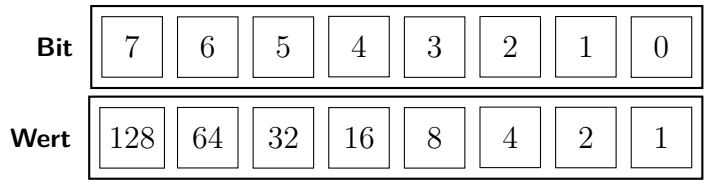


Abbildung 2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$\begin{aligned} &00000000 \rightarrow 0\,, \\ &00000011 \rightarrow 3\,, \\ &01011000 \rightarrow 88\,, \\ &11111111 \rightarrow 255\,. \end{aligned}$$

Das Addieren funktioniert nun genauso, wie ihr es aus der Schule, in der natürlich das Zehnersystem verwendet wird, gelernt habt. Beispiel:

```
  00100111        39
+ 00000110      +  6
     11           1      (Übertrag)
 ---------       ---
  00101101        45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =      10·42

 00000000
  00000000
   00000000
    00000000
     00101010
      00000000
       00101010
        00000000
         1 1                (Übertrag)
---------------
       110100100           420
```

Wenn ihr genau hinseht, dann bemerkt ihr, dass wir zwei Bytes multipliziert haben, aber das Ergebnis benötigt ein Bit mehr. Das führt zu dem Problem, dass, wenn ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte ‘‘1’’ links) abgeschnitten wird und das Ergebnis, dass der Computer euch liefert, 164 ist. Das gleich passiert bei einem Byte, wenn ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0!

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn 255 + 1 zum Ergebnis 0 hat, dann hat 0 − 1 als Ergebnis −1, binär also 11111111. Dies nennt man das ‘‘Zweierkomplement’’ (den Begriff müsst ihr euch natürlich nicht merken). Wichtig ist: dass höchste Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        −128
10000001        −127
...
11111111        −   1
00000000            0
00000001            1
...
01111111          127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
(−127)   10000001   ->   01111110
                        +00000001
                         --------
                         01111111      (127)
```

2 Hallo Welt!

In alter Tradition ist das erste Programm, das man zu sehen bekommt, wenn man eine neue Programmiersprache lernt, gewöhnlich das ‘‘Hallo Welt!’’ Programm [1]. Da wir uns zunächst auf die Programmiersprache ‘‘C’’ beschränken, ist das:

```c
/* Das ‘‘Hallo Welt’’-Programm. */
#include <stdio.h>

int main() {
  printf("Hallo Welt!\n");
  return 0;
}
```

Zeile für Zeile erklärt

Schauen wir uns das Programm Schritt für Schritt an:

`/* Das “Hallo Welt”-Programm. */` Das ist ein **Kommentar**. Der Computer ignoriert ihn völlig. Er ist nur für uns Menschen da, um den Code zu erklären.

In C gibt es zwei Arten von Kommentaren:

- `/* ... */` — für längere Kommentare über mehrere Zeilen.
- `// ...` — für kurze Kommentare in einer Zeile.

`#include <stdio.h>` Das ist eine **Header-Datei** (auf Deutsch auch *Kopfdatei*).

`stdio.h` steht für *Standard Input Output*. Sie enthält Informationen über Funktionen wie `printf`, die wir später benutzen wollen.

Merke: Wenn wir eine Funktion benutzen wollen, die wir nicht selbst geschrieben haben, müssen wir die dazugehörige Header-Datei einbinden.

`int main() {` Das ist die **Hauptfunktion** (englisch: *main function*).

Jedes C-Programm startet hier. Der Computer führt den Code in den geschweiften Klammern `{ ... }` aus.

`int` bedeutet, dass die Funktion eine ganze Zahl (Integer) zurückgibt — das ist der Rückgabewert, der weiter unten kommt.

`printf("Hallo Welt!\n");` Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

Wichtig: Jede Anweisung in C endet mit einem Semikolon ;.

`return 0;` Das ist die **Rückgabe**.

Sie sagt dem Betriebssystem: *‘‘Alles ist gut gelaufen.’’*

`0` bedeutet in der Regel *‘‘kein Fehler’’*. Wenn ein Programm mit `return 0;` endet, weiß der Computer, dass das Programm erfolgreich war.

} Die schließende geschweifte Klammer beendet die main-Funktion. Alles zwischen { und } gehört zur Funktion.

Zusammenfassung

- Ein C-Programm beginnt immer in der main()-Funktion.
- printf() schreibt Text auf den Bildschirm.
- \n macht einen Zeilenumbruch.
- return 0; sagt dem Computer: ‘‘Alles OK.’’
- Jede Anweisung endet mit ;.

Aufgaben

1. Ändere das Programm so, dass es “Guten Tag!” ausgibt.
2. Ändere das Programm so, dass es deinen Namen ausgibt.
3. Ändere das Programm so, dass es zwei Zeilen ausgibt:

   Hallo!
   Welt!

4. (Für Profis) Was passiert, wenn du das \n weglässt?

3 Zahlen

```c
#include <stdio.h>

int main() {
  int zahl = 42;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

4 Fibonacci

```c
#include <stdio.h>

int main() {
  int a = 0;
  int b = 1;

  for(int i = 1; i <= 16; i = i+1) {
    int next = a + b;
    a = b;
    b = next;

    printf("Fibonacci %d: %d\n", i, a);
  }
  return 0;
}
```

5 Reihen

```
#include <stdio.h>

int main() {

  return 0;
}
```

A Software

Als Programmierumgebung wurde VSCode verwendet. Zusätzlich ist es empfehlenswert, die Erweiterung ‘‘C C++ Toolkit’’ zu installieren.

B Leitfaden

B.1 Tastenkombinationen

Tastaturen haben ein paar besondere Tasten, die es einem erlauben, zum Beispiel Text zu markieren, kopieren und einzufügen. Auch lassen sich dadurch einige spezielle Zeichen, die oft bei der Programmierung verwendet werden, die auf einer deutschen Tastatur keine eigenen Tasten belegen, eingeben. Die Tasten, die ihr dafür benötigt, sind:

Shift Die sogenannte ‘‘Umschalttaste’’.[1] Sie schaltet zwischen Klein- und Großschreibung, sowie zwischen den verschiedenen abgebildeten Symbolen um.

Feststelltaste Im Englischen ‘‘Caps lock’’, sie befindet oberhalb der ‘‘Shift’’-Taste, sie ist dafür gedacht, dauerhaft zwisch Groß- und Kleinbuchstaben umzuschalten.[2]

Strg Die Steuerungstaste (im Englischen ‘‘Control’’ bzw. abgekürzt ‘‘Ctrl’’).

Alt ‘‘Alt’’ steht für ‘‘Alternative’’. Sie wird häufig auch in Verbindung mit ‘‘Shift’’ und ‘‘Strg’’ verwendet. Allein wird sie z.B. verwendet, um zwischen Fenstern mit ‘‘Alt+Tab’’ (‘‘Tab’’ siehe weiter unten) hin- und herzuschalten.

AltGr ‘‘AltGr’’ steht für ‘‘Alternative Grafik’’, ist im Prinzip die ‘‘rechte Alt-Taste’’, hat aber auch die Funktion, Zeichen einzugeben, die auf, z.B. einer deutschen Tastatur, sonst nicht zur Verfügung stehen.

Esc Das ist die sogenannte ‘‘escape’’-Taste, auf Deutsch übersetzt ‘‘Flucht’’. Mit dieser Taste lassen sich viele Vorgänge abbrechen.

Cursor Damit sind die Tasten gemeint, mit den man nach rechts, links, oben unten geht. Im Deutschen werden sie auch ‘‘Kursor’’, ‘‘Schreibmarke’’, ‘‘Eingabezeiger’’ usw. bezeichnet, oder einfach als ‘‘Pfeiltasten’’.

Tab Die ‘‘Tabulator’’-Taste dient dazu, Text richtig einzurücken.

B.1.1 Sonderzeichen

Die folgenden Sonderzeichen gelten für eine deutsche Tastatur:

Tastenkombination	Zeichen	Bedeutung
AltGr + Q	@	At-Zeichen
AltGr + 7	{	Geschweifte Klammer auf
AltGr + 8	[	Eckige Klammer auf
AltGr + 9	]	Eckige Klammer zu
AltGr + 0	}	Geschweifte Klammer zu
AltGr + ß	\	Backslash
AltGr + <	|	Pipe

[1] Sucht mal auf https://youtube.com nach «Umschalttaste mechanische Schreibmaschine», wenn ihr wissen wollt, woher dieser Begriff kommt.

[2] Ihr solltet im Hinterkopf behalten, dass diese Taste selten benutzt wird und auch umgedeutet werden kann. Der Autor verwendet diese Taste auf einer Englischen Tastatur, um komplexe Zeichen einzugeben. Das wird als ‘‘Compose-Taste’’ bezeichnet, z.B. ergibt bei ihm die Eingabe: CapsLock+”+a das Zeichen ‘‘ä’’, oder CapsLock+s+s das Zeichen ‘‘ß’’.

Tastenkombination	Zeichen	Bedeutung
AltGr + 3	#	Hashtag
AltGr + +	~	Tilde

B.1.2 Referenz

Die folgende Liste von Tastenkombination gilt für viele, aber nicht für alle Programme:

Tastenkombination	Bedeutung
Markieren	
Esc	Markierung aufheben
Shift+Cursor	Text markieren (alternativ mit Maus)
Strg+A	Alles auswählen (‘‘select all’’)
Kopieren und Einfügen	
Strg+C	Markierten Text kopieren
Strg+X	Markierten Text ausschneiden
Strg+V	Text einfügen
Strg+Z	Eingabe rückgängig machen (‘‘undo’’)
Häufige Funktionen	
Strg+S	Speichern (immer wieder drücken!)
Strg+F	Suchen (‘‘find’’)[3]
Strg+H	Suchen und ersetzen (‘‘replace’’)

B.2 C

B.2.1 Grundlagen

Datentypen

Ganzzahlen

Fließkommazahlen

Strukturen

Schleifen

Funktionen

Schleifen

[3] Ihr werdet vielleicht schon einmal in einem Meme ‘‘Ctrl+F’’ gesehen haben. Das bezieht sich in der Tat auf diese Tastenkombination, die allgemein ‘‘suchen’’ bedeutet

Referenz
printf

Syntax:

```
#include <stdio.h>
int printf(const char *format, ...);
```

Beschreibung:
printf () schreibt formattierten Text in die Standardausgabe (stdout). Der format-String enthält:

- Normale Zeichen (werden direkt ausgegeben)
- Format-Specifiers (beginnen mit %)

Die wichtigsten Format-Specifiers:

%d	Ganzzahl (dezimal)
%f	Gleitkommazahl
%c	Zeichen (char)
%s	Zeichenkette (String)
%x	Hexadezimal (klein)
%X	Hexadezimal (groß)
%p	Zeiger (Pointer)

Beispiel:

```
#include <stdio.h>

int main() {
    int zahl = 42;
    printf("Die Zahl ist: %d\n", zahl);
    printf("Adresse: %p\n", (void*)&zahl);
    return 0;
}

```

Rückgabewert: Die Anzahl der ausgegebenen Zeichen, oder −1 bei Fehler.

Siehe auch: scanf (), sprintf (), fprintf ()

Literatur

[1] Wikipedia. Hallo-Welt-Programm, 2003.